Introduction

PowerShell est un shell et un langage de script créé par Microsoft pour automatiser la gestion des systèmes Windows et des serveurs. Contrairement aux shells classiques, PowerShell travaille avec des **objets** plutôt que du texte.

---

1. Variables

- Une variable commence toujours par `$`.
- Elles peuvent contenir n'importe quel type de donnée : nombre, texte, tableau, objet.

Exemple

```
$prenom = "Eddy"
$age = 20
Write-Host "Je m'appelle $prenom et j'ai $age ans"
```

---

2. Types de données courants

Type	Exemple
Texte	"Bonjour"
Entier	25
Décimal	3.14
Booléen	$true / $false
Tableau	@("A","B","C")

---

3. Conditions

- `if` permet d'exécuter un bloc seulement si une condition est vraie.
- `else` ou `elseif` gère les autres cas.

Exemple

```
$age = 20
if ($age -ge 18) {
    Write-Host "Majeur"
} else {
    Write-Host "Mineur"
}
```

Opérateurs courants

Opérateur	Signification
-eq	égal
-ne	différent
-gt	supérieur à
-lt	inférieur à
-ge	supérieur ou égal
-le	inférieur ou égal
-and	ET logique
-or	OU logique
-not	NON logique

---

4. Boucles

foreach — parcourir un tableau ou une collection

```
$fruits = @("Pomme","Banane","Orange")
foreach ($fruit in $fruits) {
    Write-Host $fruit
}
```

for — compteur classique

```
for ($i=1; $i -le 5; $i++) {
    Write-Host $i
}
```

while — tant qu'une condition est vraie

```
$x = 0
while ($x -lt 5) {
    Write-Host $x
    $x++
}
```

---

5. Tableaux

- Définition : `@()`

- Accès : `$tableau[index]`
- Ajouter : `$tableau += valeur`

Exemple

```
$notes = @(10, 15, 18)
Write-Host $notes[0]   # affiche 10
$notes += 20
```

---

6. Cmdlets

PowerShell utilise des **cmdlets** : commandes pré-construites.

Format : `Verbe-Nom`

Cmdlet	Usage
Get-Service	Liste les services Windows
Get-Process	Liste les processus
Get-Content	Lire un fichier
Set-Content	Écrire dans un fichier
Copy-Item	Copier un fichier
Remove-Item	Supprimer un fichier
Start-Service	Démarrer un service
Stop-Service	Arrêter un service

---

7. Pipeline

Le pipeline `|` envoie la sortie d'une commande vers une autre commande.

```
Get-Service | Where-Object { $_.Status -eq "Running" }
```

- `$_` = objet courant dans la boucle.

---

8. Objets

- PowerShell manipule **des objets**, avec des **propriétés** (informations) et des **méthodes** (actions).
- Exemple : services, processus, fichiers, dates.

Exemple

```
$service = Get-Service -Name Spooler
$service.Status      # propriété
$service.Stop()      # méthode
```

Autre exemple avec date :

```
$date = Get-Date
$date.Year
$date.AddDays(7)
```

---

9. Fichiers et CSV

- Lire un fichier texte : `Get-Content "fichier.txt"`
- Lire un CSV :

```
$csv = Import-Csv "fichier.csv" -Delimiter ";"
foreach ($line in $csv) {
    Write-Host $line.Nom $line.Age
}
```

---

10. Fonctions

Définir une fonction :

```
function Dire-Bonjour {
    Write-Host "Bonjour"
}
```

Avec paramètre :

```
function Dire-Bonjour($nom) {
    Write-Host "Bonjour $nom"
}
```

Appel : `Dire-Bonjour "Eddy"`

---

11. Classes (niveau avancé)

```
class Etudiant {
    [string]$Nom
```

```
    [int]$Age

    DireBonjour() {
        return "Bonjour"
    }
}
```

---

12. Paramètres de script

```
param(
    [string]$Nom
)
Write-Host "Bonjour $Nom"
```

Exécution : `.\script.ps1 -Nom Eddy`

---

13. Gestion des erreurs

Ignorer les erreurs :

```
-ErrorAction SilentlyContinue
```

---

14. Affichage

Commande	Usage
Write-Host	Afficher à l'écran
Write-Output	Sortie standard
Write-Error	Afficher une erreur

---

15. Comparaison texte

```
-like "*mot*"
-match "regex"
```

---

16. Structure type d’un script

```
param()
$variables
Import-Csv / Get-Content
foreach
if
[PSCustomObject]
Write-Host
```

---

17. Conseils pour débuter

- Pratiquez chaque section avec de petits scripts
- Testez la lecture et écriture de fichiers
- Familiarisez-vous avec les cmdlets et objets
- Utilisez le pipeline pour filtrer et manipuler les données
- Refaites les exercices pour maîtriser les concepts